

On this page

Use application specific databases

Pulse supports using separate databases for application-specific tables, optimizing performance by distributing queries and storage across multiple databases. This setup provides effective isolation between applications and improves overall system efficiency.

Overview

In this guide we will do an overview on how to configure application specific databases with the assumptions:

- The deployment includes Outbound and Inbound Payments with a database for each (DB1 for Outbound and DB2 for Inbound)
- The Outbound Payments database was chosen to hold all applications' metadata

In practice, the result of these assumptions means DB2 will hold only the Event Storage table, where all events Inbound events will be stored. On the other hand DB1 will store application metadata for Outbound and Inbound Payments' applications and also the Outbound Payments Event Storage.

In the following section we will demonstrate how to configure a dedicated database for each application.

Configuring application-specific databases



Ensure that the required databases are created with sufficient CPU resources, storage, and the necessary permissions replicated from the base database. Confirm that firewall rules are configured to allow Pulse to connect to each specific database.

Also ensure that each database has adequate maximum connections available. While application-specific databases typically require fewer connections, it is recommended to set the connection limits similar to those of the base database to ensure smooth operation.

As an example consider the following:

- We have deployed two databases to support the two applications to deploy. Each database is using a proxy running locally where port 3307 connects to the outbound/metadata (DB1) database and port 3308 to the inbound one (DB2)
- For simplicity we will consider there is no read only database in the setup
- Both applications will be deployed in the same deployment process
- Both applications are for the UK market

The first step is to create a new inventory for the deployment as described in [Deploying a new application in the UK](#), where any specific variables for the deployment will be defined.

Since both applications are to be deployed in the same deployment, we need to add application specific variables. Without it both applications would use the same values to define their database connections. To achieve this we will change `project-qa/pulse/group_vars/pulse/variables.yml` as follows:

```
# Define applications to deploy
apps_enabled:
  - uk_outbound_payments
  - uk_reference_data
  - uk_inbound_payments
```

```
# Set metadata jdbc configuration
database_pulse_metadata_hostname: "localhost"
database_pulse_metadata_port: "3307"

# Set outbound jdbc configuration
# Take the same value as the metadata configuration since they will share the same database
database_pulse_eventstorage_hostname: "{{ database_pulse_metadata_hostname }}"
database_pulse_eventstorage_port: "{{ database_pulse_metadata_port }}"

# Set inbound jdbc configuration
database_pulse_inbound_payments_eventstorage_hostname: "localhost"
database_pulse_inbound_payments_eventstorage_port: "3308"
```

Copy

Notice two things in the example above:

- By default channel based variables point to the default (without channel) variables
- Since we want to define a different Inbound Payments database connection string we need to override its respective variables. These follow the syntax `database_pulse_<channel>_eventstorage_<variable_name>` as you can see above



For versions earlier than 2.7.0 channel based variables are defined in the format `<app_name>_<variable_name>`, where `<app_name>` is the application name as defined in `apps_enabled`.

In the context of this guide, we would configure the connection string as follows:

```
# Set inbound jdbc configuration
uk_inbound_payments_database_pulse_eventstorage_hostname: "localhost"
uk_inbound_payments_database_pulse_eventstorage_port: "3308"
uk_inbound_payments_database_pulse_eventstorage_jdbc: "jdbc:postgresql://"
{{ uk_inbound_payments_database_pulse_eventstorage_hostname }}:
{{ uk_inbound_payments_database_pulse_eventstorage_port }}/
{{database_pulse_eventstorage_database_name }}?{{ database_pulse_eventstorage_jdbc_opts |
join('&') }}"
```

Copy

The deployment would continue by issuing the installation command:

```
python3.6 ansible-playbook \
-i project-default \
-i project-hsbc \
-i gcp \
-i project-qa/uk \
pulse.yml
-e deployment_color=<color to deploy>
```

Copy

Assuming the deployment ran without issues, you should be able to see the configuration in the Pulse configuration page as illustrated below.



We would not need to define the Inbound variables as `<app>_<variable_name>` if Inbound was deployed on a different Pulse node. We could instead have the default variables defined (as we did for Outbound) and pass a application-specific inventory during deployment



Find a detailed description of all available configurations in [Pulse Configuration Recommendations](#)